Niyukti: Raised Problems

Problem: “If a recruiter specifies ‘Must know Microservices and Spring Boot’, but a candidate only writes ‘Built REST APIs using Java’ in their resume, how does your matching engine determine true semantic alignment without hallucinating experience?”

Critical Assessment

A resume is a self-written claim, so Niyukti verifies **whether the resume contains sufficient, quotable evidence** for each mandatory requirement.

Identified Weakness	Proposed Correction
1. Semantic similarity alone may produce incorrect verification decisions.	Semantic matching and embeddings will be used to identify potentially relevant resume passages, not to determine the final outcome. A deterministic rule engine will evaluate the retrieved evidence against predefined verification criteria.
2. An LLM may generate unsupported skill claims or fabricated evidence.	Each extracted skill must be linked to an exact quotation from the parsed resume and its source location. The system will validate that the quotation exists and check its context and relevance before accepting it as supporting evidence.
3. The minimum evidence required to verify a skill is undefined.	Define evidence levels and requirement-specific acceptance rules. For example, an explicit skill mention may be sufficient for a basic mention requirement, whereas a practical-experience requirement may require relevant project evidence or an assessment. The T&P team can configure the applicable policy for each role.
4. Related technologies may be incorrectly treated as equivalent.	Maintain a controlled skill taxonomy with typed relationships. ALIAS identifies equivalent terms, while RELATED_TO identifies related but distinct technologies. Related skills cannot independently satisfy a mandatory requirement. Any IMPLIES rule must be explicitly justified and approved.

Proposed Solution

Core Principle: Retrieve semantically, decide deterministically, and verify with evidence.

Niyukti uses a structured, evidence-based approach to match candidate resumes against Job Description (JD) requirements while minimizing unsupported skill claims.

First, AI extracts structured information from the JD and resume, identifying required skills, eligibility criteria, and relevant experience. Each extracted skill must be linked to a supporting quotation from the resume, which is checked against the source text.

Next, a controlled skill taxonomy distinguishes equivalent terms (aliases) from related but distinct technologies. For example, Java and Spring Boot are related technologies, but evidence of Java alone cannot establish Spring Boot experience.

A deterministic rule engine then evaluates the validated evidence against predefined requirements and returns one of two final statuses: **Verified** or **Rejected**, along with a clear explanation for the decision. A rejection indicates insufficient evidence or unmet defined criteria in the submitted information; it does not necessarily mean the candidate lacks the skill.

Finally, the T&P team retains authority over placement decisions and can review cases where evidence is unclear or requires further verification. This approach aims to make skill matching more explainable, auditable, and practical to implement within our 12-member team.

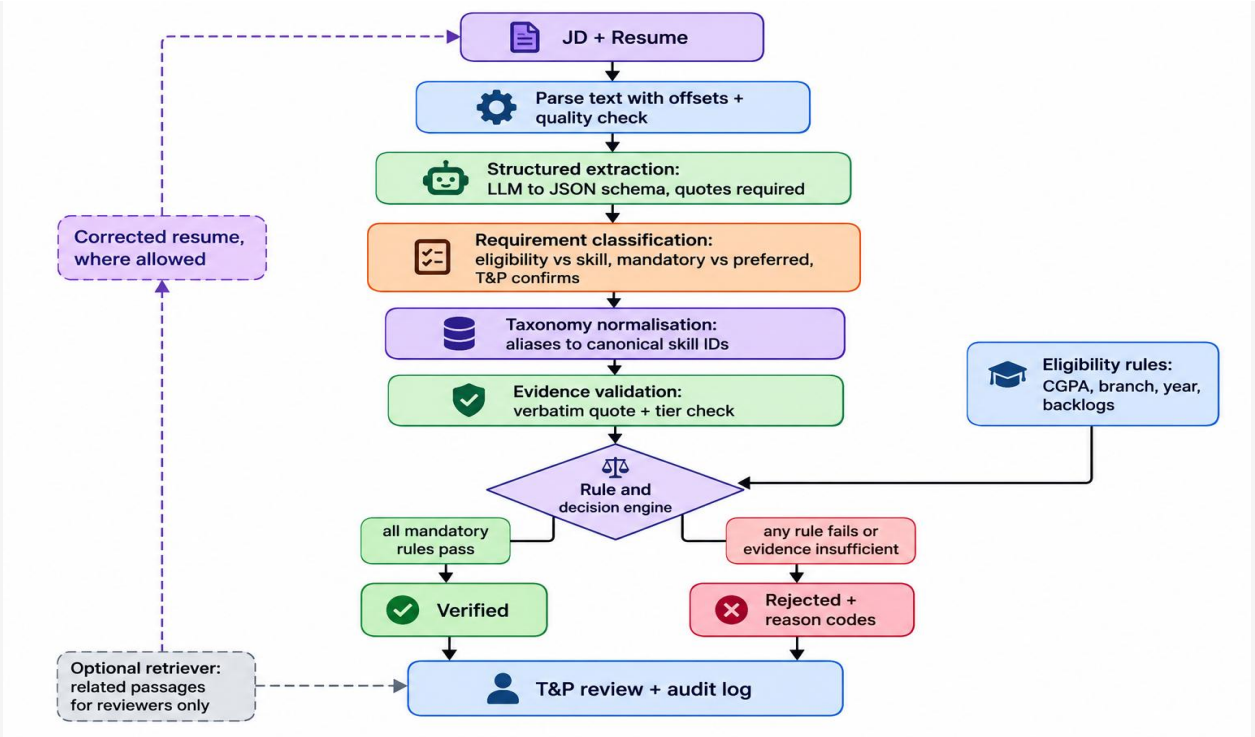
Skill Taxonomy and Semantic Matching

Relation	Example	Can satisfy a requirement?
`ALIAS` (same skill)	JS = JavaScript; "micro-services" = Microservices	Yes
`IMPLIES` (directed, whitelisted)	Spring Boot → Java	Yes, that direction only
`RELATED`	Java ~ Spring Boot; REST API ~ Microservices	No, context only

Related skills are not equivalent because direction matters (Spring Boot developers know Java, not vice versa), and REST is an interface style while microservices is an architecture: a monolith can expose REST

APIs. Semantic similarity is used only to show reviewers nearby passages and to suggest new aliases for maintainers to approve. A high score is a reason to *look*, never to *credit*.

Architecture:



Detailed Example

JD: "Must know Microservices and Spring Boot" gives mandatory `spring_boot` and `microservices`.
Resume (Projects): "Built REST APIs using Java". Assume eligibility passes.

Skill	Mandatory?	Resume evidence	Result
Java	No	"Built REST APIs using Java"	Supported
REST API development	No	Same sentence	Supported
Spring Boot	Yes	None naming it or an alias	**Insufficient evidence**
Microservices	Yes	None naming it or an alias	**Insufficient evidence**

The retriever may show the sentence as "related, not counted"; it changes nothing. **Result: Rejected**, reason codes `SKILL_EVIDENCE_NOT_FOUND` for both skills. The explanation states that related evidence (Java, REST) is not sufficient under the rules, that this does not prove the candidate lacks the skills, and that re-verification is possible after a resume correction. If the resume later says "Developed three Spring Boot microservices communicating via REST", the same rules yield **Verified** with quotable evidence.

Hallucination Prevention

- **[MVP]** Mandatory verbatim quotes with offsets, checked programmatically; invalid JSON or unverifiable items rejected.
- **[MVP]** Typed taxonomy with whitelisted `IMPLIES` only; decision engine is plain code; explanations are templated from the rule trace.
- **[MVP]** Resume treated as untrusted data (basic injection defense); fixed model version and temperature 0; audit log of inputs, versions, policy and output.
- **[Design]** T&P review queue and override.
- **Limits:** these stop the system *inventing* evidence. They cannot detect a candidate's false claim, and they do not remove missed evidence caused by unusual wording.

Evaluation and Testing

No results are claimed; this is the planned method. Build a labelled set of JD/resume pairs (consented or synthetic, about 60 to 100 as capacity allows) with edge cases: aliases, related-not-equal skills, hedging, and skills-only lists. Two annotators label whether each mandatory skill has sufficient evidence; a third resolves disagreements and agreement (Cohen's kappa) is reported. Keep a hold-out set.

- **Positive class:** evidence present. **False positive (critical):** unsupported credit, e.g. Java treated as Spring Boot.
- **False negative:** real evidence missed, causing wrongful rejection. Report precision and recall per skill and per final status; the primary goal is zero false positives on related-but-not-equal cases.
- **Baselines:** compare keyword-only, embedding threshold, LLM-only and Niyukti on the same set, reporting results honestly.